Τμήμα Πληροφορικής & Τηλεπικοινωνιών
Τεχνολογίες Εφαρμογών Διαδικτύου

NexusTickets

Εφαρμογή Διαχείρισης Εκδηλώσεων και
Ηλεκτρονικών Κρατήσεων στον Παγκόσμιο Ιστό

Μάριος Γιαννόπουλος
ΑΜ: 1115202000032

Ραμαντάν Κονόμι
ΑΜ: 1115201800281

Αθήνα, Σεπτέμβριος 2026

Πίνακας Περιεχομένων

1 Εισαγωγή 1
1.1 Δομή της Αναφοράς 1
2 Αρχιτεκτονική Συστήματος 2
2.1 Επισκόπηση 2
2.2 Ροή Αιτήματος 2
2.3 Ροή Πραγματικού Χρόνου 3
2.4 Εργασίες Παρασκηνίου 3
2.5 Ανάπτυξη και Εκτέλεση 3
3 Σχεδιασμός Βάσης Δεδομένων 3
3.1 Διάγραμμα Οντοτήτων–Συσχετίσεων 4
3.2 Πίνακες 4
3.3 Δείκτες 6
3.4 Redis 6
4 Υλοποίηση Backend 7
4.1 Δομή Έργου 7
4.2 Διαμόρφωση (Configuration) 7
4.3 Βάση Δεδομένων 8
4.4 Επίπεδο Τομέα (Domain Layer) 9
4.5 Επίπεδο Αποθετηρίου (Repository Layer) 10
4.6 Επίπεδο Υπηρεσιών (Service Layer) 12
4.7 Επίπεδο Χειριστών (Handler Layer) 16
4.8 Ενδιάμεσο Λογισμικό (Middleware) 18
4.9 WebSocket 20
4.10 Εξαγωγή XML και JSON 21
4.11 Έγχυση Εξαρτήσεων (Dependency Injection) 21
5 Υλοποίηση Frontend 22
5.1 Δομή Έργου 22
5.2 Διεθνοποίηση 22
5.3 Διαχείριση Κατάστασης 23
5.4 Αυθεντικοποίηση στον Πελάτη 23
5.5 Σελίδες και Απαιτήσεις 23
5.6 Η Θυρίδα Μηνυμάτων 24
5.7 Ειδοποιήσεις Πραγματικού Χρόνου 25
6 Αυθεντικοποίηση και Ασφάλεια 25
6.1 Κρυπτογράφηση Μεταφοράς (TLS) 25
6.2 Αυθεντικοποίηση με JSON Web Tokens 25
6.3 Εξουσιοδότηση ανά Ρόλο 26

6.4 Λοιπά Μέτρα 26
7 Σύστημα Συστάσεων 27
7.1 Ο Αλγόριθμος BMF 27
7.2 Δεδομένα Εκπαίδευσης — Έμμεση Ανατροφοδότηση 27
7.3 Παρουσίαση Συστάσεων 28
7.4 Αξιολόγηση με το Παρεχόμενο Σύνολο Δεδομένων 28
8 Δοκιμές 28
8.1 Backend 28
8.2 Frontend 29
8.3 Στατικοί Έλεγχοι και Ολοκλήρωση 30
9 Επίλογος 30
9.1 Δυσκολίες και Λύσεις 30
9.2 Αποτίμηση 31

1 Εισαγωγή

Η παρούσα εργασία αφορά τον σχεδιασμό και την υλοποίηση της εφαρμογής **NexusTickets**, μιας διαδικτυακής πλατφόρμας διαχείρισης εκδηλώσεων και ηλεκτρονικών κρατήσεων εισιτηρίων, στο πλαίσιο του μαθήματος «Τεχνολογίες Εφαρμογών Διαδικτύου». Η εφαρμογή ακολουθεί την αρχιτεκτονική web browser / web server: το μετωπιαίο άκρο (frontend) είναι μια εφαρμογή μίας σελίδας (Single-Page Application) σε React/ TypeScript, ενώ το νωτιαίο άκρο (backend) είναι μια υπηρεσία REST σε Go, η οποία αντιστοιχίζει το σχεσιακό μοντέλο της βάσης PostgreSQL σε αντικειμενοστρεφές μοντέλο και παρέχει επιπλέον ένα κανάλι WebSocket για ειδοποιήσεις πραγματικού χρόνου.

Η εφαρμογή υποστηρίζει τους τέσσερις ρόλους της εκφώνησης. Ο **Επισκέπτης** πλοηγείται και αναζητά εκδηλώσεις χωρίς να μπορεί να κάνει κράτηση. Ο **Συμμετέχων** κάνει κρατήσεις, επικοινωνεί με τους διοργανωτές των εκδηλώσεων που έχει κρατήσει και λαμβάνει εξατομικευμένες προτάσεις. Ο **Διοργανωτής** δημιουργεί, δημοσιεύει, τροποποιεί, ακυρώνει και διαγράφει εκδηλώσεις, ορίζει χωρητικότητα και τύπους εισιτηρίων, επισκοπεί τις κρατήσεις τους και ενημερώνει μαζικά τους συμμετέχοντες. Ο **Διαχειριστής**, ενσωματωμένος από την εγκατάσταση, εγκρίνει ή απορρίπτει τις αιτήσεις εγγραφής, διαχειρίζεται χρήστες και εκδηλώσεις και εξάγει τον κατάλογο εκδηλώσεων σε XML (σύμφωνα με το DTD της εκφώνησης) και JSON.

Ιδιαίτερη βαρύτητα δόθηκε σε τρία σημεία: στην **ορθότητα υπό ταυτόχρονη πρόσβαση** (η δέσμευση θέσεων γίνεται με μία ατομική ενημέρωση στη βάση, ώστε να μην είναι δυνατή η υπερκράτηση), στην **ασφάλεια** (TLS σε όλες τις αλληλεπιδράσεις, αυθεντικοποίηση με JSON Web Tokens, εξουσιοδότηση ανά ρόλο σε κάθε διαδρομή) και στο **σύστημα συστάσεων**, όπου ο αλγόριθμος Biased Matrix Factorization υλοποιήθηκε εκ του μηδενός και εκπαιδεύεται στα πραγματικά δεδομένα κρατήσεων και προβολών της εφαρμογής.

1.1 Δομή της Αναφοράς

- Το **Κεφάλαιο 2** παρουσιάζει τη συνολική αρχιτεκτονική του συστήματος και τις βασικές τεχνολογικές επιλογές.
- Το **Κεφάλαιο 3** περιγράφει τον σχεδιασμό της σχεσιακής βάσης δεδομένων και την αντιστοίχισή της στο DTD της εκφώνησης.
- Το **Κεφάλαιο 4** αναλύει την υλοποίηση του backend, επίπεδο προς επίπεδο.
- Το **Κεφάλαιο 5** περιγράφει το frontend: δομή, διαχείριση κατάστασης, σελίδες και τη σύνδεσή τους με τις απαιτήσεις.
- Το **Κεφάλαιο 6** συγκεντρώνει τα θέματα αυθεντικοποίησης και ασφάλειας.
- Το **Κεφάλαιο 7** παρουσιάζει τον αλγόριθμο συστάσεων και την αξιολόγησή του.
- Το **Κεφάλαιο 8** περιγράφει τη στρατηγική δοκιμών.

- Ο **Επίλογος** συνοψίζει την πορεία ανάπτυξης, τις δυσκολίες που αντιμετωπίστηκαν και τον τρόπο επίλυσής τους.

2 Αρχιτεκτονική Συστήματος

Το κεφάλαιο περιγράφει τα μέρη του συστήματος, τον τρόπο που επικοινωνούν και τις εργασίες που τρέχουν στο παρασκήνιο, πριν η ανάλυση προχωρήσει στη βάση δεδομένων και στα δύο άκρα της εφαρμογής.

2.1 Επισκόπηση

Το σύστημα αποτελείται από τέσσερα τρέχοντα μέρη, τα οποία ορχηστρώνονται με Docker Compose:

Μέρος	**Τεχνολογία**	**Ρόλος**
Frontend	React 19, TypeScript, Vite, Tailwind CSS v4, shadcn/ui, Redux Toolkit / RTK Query, React Router v7, react-leaflet	SPA · όλη η διεπαφή χρήστη
Backend	Go 1.22, Fiber v2, GORM, Wire, gofiber/websocket, Zap	REST API, WebSocket push, επιχειρηματική λογική, recommender
Βάση δεδομένων	PostgreSQL 16	Μόνιμη αποθήκευση· full-text search στα ελληνικά
Cache / session store	Redis 7	Refresh tokens, rate limiting, μετρητές μη αναγνωσμένων

Πίνακας 1: Τα μέρη του συστήματος

Ο browser επικοινωνεί αποκλειστικά με το backend μέσω HTTPS: με κλήσεις REST (JSON) για κάθε λειτουργία και με μία σύνδεση WebSocket (`wss://`) για τη λήψη ειδοποιήσεων. Το backend είναι η μοναδική είσοδος στα δεδομένα· ο browser δεν έχει καμία απευθείας πρόσβαση στη βάση.



Σχήμα 1: Τα μέρη του συστήματος και οι διαδρομές επικοινωνίας τους

2.2 Ροή Αιτήματος

Κάθε αίτημα REST διασχίζει τα ίδια στρώματα: το ενδιάμεσο λογισμικό (CORS, request id, καταγραφή, rate limiting, αυθεντικοποίηση JWT, έλεγχος ρόλου), τον handler (ανά-

λυση και επικύρωση του DTO), την υπηρεσία (επιχειρηματικοί κανόνες) και το αποθετήριο (GORM). Οι υπηρεσίες επιστρέφουν sentinel errors του domain, τα οποία ένας κεντρικός `ErrorHandler` αντιστοιχίζει άπαξ σε HTTP κωδικούς. Όλες οι αποκρίσεις έχουν την ίδια μορφή envelope `{data, error, meta}`, ώστε ο πελάτης να τις αποσυσκευάζει με έναν κοινό μηχανισμό.

2.3 Ροή Πραγματικού Χρόνου

Η επικοινωνία διοργανωτή–συμμετέχοντα (Απαίτηση 10) αποθηκεύεται πάντα μέσω REST. Το WebSocket χρησιμοποιείται μόνο ως κανάλι **ώθησης** από τον server προς τον πελάτη: όταν ένα μήνυμα αποθηκευτεί, το Hub το προωθεί σε όλες τις ανοιχτές συνδέσεις του παραλήπτη μαζί με τον νέο μετρητή μη αναγνωσμένων. Έτσι το σήμα «έχετε νέο μήνυμα» εμφανίζεται άμεσα στη γραμμή πλοήγησης, όπου κι αν βρίσκεται ο χρήστης, χωρίς polling.

2.4 Εργασίες Παρασκηνίου

Το backend εκκινεί τρεις goroutines: τον βρόχο του WebSocket Hub, την περιοδική επανεκπαίδευση του μοντέλου συστάσεων (στην εκκίνηση και κάθε 6 ώρες) και έναν «σαρωτή» που κάθε 15 λεπτά μεταφέρει τις δημοσιευμένες εκδηλώσεις των οποίων η ώρα λήξης έχει παρέλθει στην κατάσταση `COMPLETED`.

2.5 Ανάπτυξη και Εκτέλεση

Σε περιβάλλον ανάπτυξης ο Vite dev server εξυπηρετεί το SPA στο `https://localhost:5173` με τοπικό πιστοποιητικό (mkcert) και προωθεί τα `/api`, `/uploads` και το WebSocket στο backend, το οποίο επίσης ακούει με TLS. Σε παραγωγικό περιβάλλον το frontend μεταγλωττίζεται σε στατικά αρχεία και εξυπηρετείται από nginx, που τερματίζει το TLS στη θύρα 443 και προωθεί τα ίδια μονοπάτια στο backend — και αυτό το εσωτερικό σκέλος ταξιδεύει κρυπτογραφημένο, αφού το backend σερβίρει το δικό του TLS και το nginx το προσεγγίζει με `proxy_pass https://`. Και στις δύο περιπτώσεις ο browser βλέπει μία μόνο προέλευση (origin), οπότε δεν απαιτούνται cross-origin αιτήματα.

3 Σχεδιασμός Βάσης Δεδομένων

Η σχεσιακή βάση σχεδιάστηκε με αφετηρία το DTD της εκφώνησης: κάθε στοιχείο `<Event>` έγινε γραμμή του πίνακα `events`, τα επαναλαμβανόμενα στοιχεία (`Category+`, `TicketType+`, `Booking*`, `Photo*`) έγιναν εξαρτώμενοι πίνακες και οι αναφορές σε χρήστες (`Organizer`, `Attendee`) έγιναν ξένα κλειδιά προς τον πίνακα `users`. Το σχήμα δηλώνεται στα GORM models (`internal/repository/postgres/models.go`) και εφαρμόζεται στην εκκίνηση από το `internal/db/automigrate.go`, το οποίο προσθέτει επιπλέον ό,τι δεν εκφράζει το GORM: περιορισμούς CHECK, ξένα κλειδιά με ρητούς κανόνες `ON DELETE`, τη γεννώμενη στήλη πλήρους κειμένου και τους δείκτες.

3.1 Διάγραμμα Οντοτήτων–Συσχετίσεων

Από	Προς	Πληθικότητα	Σημασία / κανόνας διαγραφής
users	events	1 : N	διοργανωτής της εκδήλωσης · RESTRICT
users	bookings	1 : N	ο συμμετέχων της κράτησης · RESTRICT
users	messages	1 : N (×2)	αποστολέας και παραλήπτης · RESTRICT
users	user_event_views	1 : N	προβολές για τον recommender · CASCADE
events	ticket_types	1 : N	τύποι εισιτηρίων · CASCADE
events	bookings	1 : N	κρατήσεις · RESTRICT
events	event_photos	1 : N	φωτογραφίες · CASCADE
events	categories	M : N	μέσω event_categories · κατηγορία σε χρήση δεν διαγράφεται
events	messages	1 : N (προαιρ.)	η εκδήλωση που αφορά ένα μήνυμα · SET NULL
events	user_event_views	1 : N	CASCADE
ticket_types	bookings	1 : N	ο τύπος που κρατήθηκε · RESTRICT
users	support_tickets	1 : N	αιτήματα υποστήριξης · RESTRICT
support_tickets	support_ticket_messages	1 : N	νήμα απαντήσεων · CASCADE

Πίνακας 2: Συσχετίσεις μεταξύ των πινάκων

Οι κανόνες RESTRICT στις κρατήσεις είναι σκόπιμοι: εκδήλωση, τύπος εισιτηρίου ή χρήστης με κρατήσεις δεν μπορεί να διαγραφεί φυσικά, γι' αυτό η ακύρωση εκδήλωσης είναι μεταβολή κατάστασης και όχι διαγραφή (Απαίτηση 7γ).

3.2 Πίνακες

3.2.1 users

Στήλη	Τύπος	Σημείωση
id	uuid PK	
username, email	varchar	μοναδικά
password_hash	varchar	bcrypt, κόστος 12· δεν εκτίθεται ποτέ
first_name, last_name	varchar	
phone, address, city, country	varchar, nullable	στοιχεία επικοινωνίας και διεύθυνσης (Απ. 2)
latitude, longitude	double, nullable	γεωγραφική τοποθεσία (Απ. 2)
vat_number	varchar, nullable	ΑΦΜ (9 ψηφία)
role	varchar + CHECK	admin · organizer · attendee (το CHECK δέχεται και guest, που δεν ανατίθεται ποτέ σε λογαριασμό)
account_status	varchar + CHECK	pending · approved · rejected (Απ. 2–4)

created_at, updated_at	timestamptz	

Πίνακας 3: Πίνακας users

Ο ρόλος επιλέγεται στην εγγραφή και μπορεί να αλλάξει από τον διαχειριστή. Ο επισκέπτης δεν αποθηκεύεται ποτέ: είναι απλώς η απουσία token.

3.2.2 events

Στήλη	Τύπος	DTD / σημείωση
id	uuid PK	EventID
organizer_id	uuid FK	<Organizer UserID>
title, description	varchar / text	Title, Description
venue, address, city, country	varchar	Venue, Address, City, Country
event_type	varchar	EventType
latitude, longitude	double, nullable	<GeoLocation>
start_time, end_time	timestamptz	StartDateTime, EndDateTime · CHECK end ≥ start
capacity	int	Capacity · CHECK ≥ 0
status	varchar + CHECK	draft · published · completed · cancelled· το CHECK δέχεται επιπλέον την παλαιά τιμή deleted
featured_image_url	varchar, nullable	εικόνα εξωφύλλου
event_detail	jsonb	προαιρετικό πλούσιο περιεχόμενο (εκτός DTD)
search_vector	tsvector, generated	to_tsvector('greek', title ‖ description ‖ venue ‖ city), δείκτης GIN

Πίνακας 4: Πίνακας events

3.2.3 categories και event_categories

Η ταξινομία των κατηγοριών (id, name μοναδικό) και ο πίνακας σύνδεσης M:N με σύνθετο πρωτεύον κλειδί (event_id, category_id). Το πολλαπλό <Category> του DTD παράγεται από αυτή τη σύνδεση. Νέες κατηγορίες δημιουργούνται αυτόματα όταν ένας διοργανωτής πληκτρολογήσει όνομα που δεν υπάρχει.

3.2.4 ticket_types

id, event_id, name, price numeric(12,2), quantity (συνολικό πλήθος), available (διαθέσιμο υπόλοιπο). Περιορισμοί: price ≥ 0, quantity ≥ 0, 0 ≤ available ≤ quantity, μοναδικό (event_id, name). Ο κανόνας $\sum \text{quantity} \leq \text{capacity}$ της Απαίτησης 7 επιβάλλεται στο επίπεδο υπηρεσιών σε κάθε δημιουργία ή τροποποίηση.

3.2.5 bookings

id, user_id, event_id, ticket_type_id, quantity (CHECK > 0), total_cost numeric(12,2) (υπολογίζεται από τον server), status (pending · confirmed · cancelled), created_at. Η δέσμευση θέσεων εκτελείται σε μία συναλλαγή που περιλαμβάνει την εντολή

```
UPDATE ticket_types SET available = available - $n
WHERE id = $id AND available >= $n;
```

Αν επηρεαστούν 0 γραμμές, η συναλλαγή αναιρείται και επιστρέφεται `ErrCapacityExceeded`. Επειδή ο έλεγχος και η μείωση είναι μία ατομική πρόταση SQL, δύο ταυτόχρονες κρατήσεις δεν μπορούν να υπερβούν τη διαθεσιμότητα.

3.2.6 `messages`

`id`, `sender_id`, `recipient_id`, `event_id` (nullable), `subject`, `body`, `read_at` (NULL = μη αναγνωσμένο), `deleted_by_sender`, `deleted_by_recipient`, `created_at`. Μία γραμμή ανά μήνυμα: τα εισερχόμενα του παραλήπτη και τα απεσταλμένα του αποστολέα είναι η ίδια γραμμή, ενώ οι δύο σημαίες διαγραφής επιτρέπουν σε κάθε μέρος να αφαιρέσει το μήνυμα από τον δικό του κατάλογο ανεξάρτητα. Ένας μερικός δείκτης `(recipient_id) WHERE read_at IS NULL` εξυπηρετεί τον μετρητή μη αναγνωσμένων.

3.2.7 `support_tickets` και `support_ticket_messages`

Τα αιτήματα υποστήριξης: `support_tickets` (`user_id`, `subject`, `status` με CHECK `open` · `answered` · `closed`, `created_at`, `updated_at`) και το νήμα τους `support_ticket_messages` (`ticket_id` CASCADE, `author_id`, `body`, `created_at`). Το `updated_at` του ticket ενημερώνεται σε κάθε απάντηση, ώστε η ουρά της διαχείρισης να ταξινομείται κατά πρόσφατη δραστηριότητα.

3.2.8 `event_photos` και `user_event_views`

Οι φωτογραφίες (`storage_path`, `position` μοναδικό ανά εκδήλωση) αντιστοιχούν στο `<Media><Photo>`. Οι προβολές (`user_id`, `event_id`, `viewed_at`) καταγράφονται κάθε φορά που συνδεδεμένος χρήστης ανοίγει σελίδα εκδήλωσης και τροφοδοτούν, μαζί με τις κρατήσεις, τον αλγόριθμο συστάσεων.

3.3 Δείκτες

Πέρα από τα μοναδικά κλειδιά, ορίζονται δείκτες για τα συνήθη ερωτήματα: `events(status, start_time)`, `events(city)`, `events(organizer_id)`, GIN στο `search_vector` · `bookings(user_id, created_at)`, `bookings(event_id, created_at)` · `messages(sender_id, created_at)`, `messages(recipient_id, created_at)` και ο μερικός δείκτης μη αναγνωσμένων· `user_event_views(user_id, viewed_at)` και `(event_id, viewed_at)`.

3.4 Redis

Το Redis δεν αποθηκεύει δεδομένα του πεδίου, μόνο εφήμερη κατάσταση: τα έγκυρα refresh tokens (κλειδί ανά `jti`, TTL 7 ημέρες), τους μετρητές του sliding-window rate limiter και τον μετρητή μη αναγνωσμένων μηνυμάτων ανά χρήστη, ο οποίος επανυπολογίζεται από την PostgreSQL σε κάθε μεταβολή, ώστε η βάση να παραμένει η μοναδική πηγή αλήθειας.

4 Υλοποίηση Backend

Το backend του NexusTickets υλοποιήθηκε εξ ολοκλήρου στη γλώσσα Go (έκδοση 1.22), αξιοποιώντας το πλαίσιο ανάπτυξης Fiber v2 για τη δημιουργία REST API και WebSocket διακομιστή. Η αρχιτεκτονική που ακολουθήθηκε είναι η Clean Architecture (Καθαρή Αρχιτεκτονική), η οποία διαχωρίζει την εφαρμογή σε τέσσερα διακριτά επίπεδα: το επίπεδο τομέα (domain layer), το επίπεδο αποθετηρίου (repository layer), το επίπεδο υπηρεσιών (service layer) και το επίπεδο χειριστών (handler layer). Ο διαχωρισμός αυτός εξασφαλίζει τη δυνατότητα δοκιμών (testability), τη συντηρησιμότητα του κώδικα και την ανεξαρτησία από εξωτερικά πλαίσια.

4.1 Δομή Έργου

Η δομή του backend ακολουθεί τις συμβάσεις της κοινότητας Go:

Κατάλογος	Περιγραφή	Ρόλος
`cmd/server/`	Είσοδος εφαρμογής	Εκκίνηση HTTP/Fiber server
`cmd/seed/`	Seeder δεδομένων	150 εκδηλώσεις, 7 κατηγορίες, χρήστες, κρατήσεις, μηνύματα, αιτήματα, φωτογραφίες
`internal/config/`	Διαμόρφωση	Φόρτωση & επικύρωση env vars
`internal/domain/`	Οντότητες & Σταθερές	Μοντέλο πεδίου
`internal/dto/`	Data Transfer Objects	Αιτήσεις & Αποκρίσεις
`internal/repository/`	Αποθετήριο	Πρόσβαση σε βάση δεδομένων
`internal/service/`	Υπηρεσίες	Επιχειρηματική λογική
`internal/handler/`	Χειριστές	REST & WebSocket endpoints
`internal/middleware/`	Ενδιάμεσο λογισμικό	Auth, Rate limiting, CORS
`internal/redis/`	Πελάτης Redis	Σύνδεση & παραμετροποίηση
`internal/export/`	Εξαγωγή	XML & JSON streaming
`internal/wire/`	Dependency Injection	Wire providers & sets
`pkg/response/`	Απάντηση API	Envelope OK/Created/Error
`pkg/pagination/`	Σελιδοποίηση	Page/Meta helpers
`pkg/logger/`	Καταγραφή	Zap structured logger
`internal/db/`	Σχήμα ΒΔ	Σύνδεση Postgres, AutoMigrate, constraints, indexes

Πίνακας 5: Δομή καταλόγων του backend

4.2 Διαμόρφωση (Configuration)

Η διαμόρφωση της εφαρμογής πραγματοποιείται μέσω μεταβλητών περιβάλλοντος (environment variables). Το πακέτο `internal/config/config.go` ορίζει τη δομή `Config` που περιέχει όλες τις ρυθμιζόμενες παραμέτρους της εφαρμογής, όπως τα στοιχεία σύνδεσης στη βάση δεδομένων, το secret για τα JWT tokens, τις ρυθμίσεις CORS, τις παραμέτρους του rate limiter και τη διαδρομή αποθήκευσης πολυμέσων.

Η φόρτωση γίνεται μέσω της συνάρτησης `Load()` η οποία:

- Καλεί την `loadDotEnv()` για προαιρετική φόρτωση αρχείου `.env` (μόνο σε περιβάλλον ανάπτυξης).
- Διαβάζει κάθε μεταβλητή περιβάλλοντος με τη συνάρτηση `getEnv(key, fallback)`.
- Αναλύει αριθμητικές παραμέτρους (ports, limits) με τις `parsePort`, `parseInt`, `parseDuration`.
- Επικυρώνει όλες τις τιμές χρησιμοποιώντας τη βιβλιοθήκη `go-playground/validator/v10`.
- Ελέγχει την ύπαρξη των αρχείων TLS (εφόσον έχουν οριστεί).

Παραδείγματα προεπιλεγμένων τιμών:

Μεταβλητή	**Προεπιλογή**	**Περιγραφή**
APP_ENV	development	Περιβάλλον εκτέλεσης
APP_PORT	8080	Θύρα HTTP server
DB_HOST	postgres	Διεύθυνση PostgreSQL
DB_MAX_OPEN_CONNS	25	Μέγιστες ταυτόχρονες συνδέσεις DB
RATE_LIMIT_MAX_REQUESTS	120	Μέγιστα requests ανά λεπτό
JWT_SECRET	dev-insecure-…	Secret για HMAC-SHA256

Πίνακας 6: Παράδειγμα μεταβλητών περιβάλλοντος και προεπιλογών

4.3 Βάση Δεδομένων

4.3.1 Σύνδεση

Το backend χρησιμοποιεί δύο αποθηκευτικά συστήματα:

1. **PostgreSQL 16** ως κύρια σχεσιακή βάση δεδομένων, με πρόσβαση μέσω GORM.
2. **Redis 7** για caching, rate limiting, αποθήκευση refresh token JTIs και μετρητές unread messages.

Η σύνδεση με την PostgreSQL γίνεται στο `internal/db/postgres.go` μέσω του GORM. Ορίζονται ρυθμίσεις για το pool συνδέσεων (`SetMaxOpenConns`, `SetMaxIdleConns`, `SetConnMaxLifetime`, `SetConnMaxIdleTime`) και πραγματοποιείται ping με timeout 3 δευτερολέπτων για επιβεβαίωση της σύνδεσης.

4.3.2 AutoMigrate

Η εξέλιξη της βάσης δεδομένων γίνεται μέσω του GORM AutoMigrate στο αρχείο `internal/db/automigrate.go`. Το AutoMigrate εκτελείται σε κάθε εκκίνηση του διακομιστή και δημιουργεί ή ενημερώνει τους πίνακες χωρίς απώλεια δεδομένων.

Μετά το AutoMigrate, εκτελούνται επιπλέον SQL εντολές με τη ρήτρα `IF NOT EXISTS` για:

- **13 CHECK constraints** — π.χ. `users_role_chk`, `events_capacity_chk`, `ticket_types_available_chk`, `bookings_quantity_chk`.

- **16 FOREIGN KEY constraints** — π.χ. `fk_events_organizer`, `fk_bookings_ticket_type`, `fk_messages_sender`, `fk_event_photos_event`.
- **2 UNIQUE composite indexes** — `ticket_types_event_name_uq` και `event_photos_event_position_uq`.
- **1 generated column** — `search_vector tsvector GENERATED ALWAYS AS (...) STORED` με Greek full-text search configuration.
- **19 performance indexes** — π.χ. `events_search_vector_gin_idx`, `events_status_start_time_idx`, `messages_recipient_unread_idx`, `ticket_types_event_id_idx`.

Ο μηχανισμός `search_vector` χρησιμοποιεί `to_tsvector('greek', coalesce(title, '') || ' ' || coalesce(description, '') || ' ' || coalesce(venue, '') || ' ' || coalesce(city, ''))` για την υποστήριξη αναζήτησης ελεύθερου κειμένου στα ελληνικά.

4.4 Επίπεδο Τομέα (Domain Layer)

Το επίπεδο τομέα βρίσκεται στο πακέτο `internal/domain/` και περιέχει τις βασικές οντότητες, τις σταθερές και τα σφάλματα της εφαρμογής.

4.4.1 Οντότητες

Οι κύριες οντότητες που ορίζονται είναι:

- **User** — Χρήστης με πεδία: ID (UUID v7), Username, PasswordHash, FirstName, LastName, Email, PhotoURL, Phone, Address, City, Country, Latitude, Longitude, VATNumber, Role, AccountStatus, CreatedAt, UpdatedAt.
- **Event** — Εκδήλωση με πεδία: ID, OrganizerID, Title, Description, Venue, EventType, FeaturedImageURL, Detail (jsonb), Address, City, Country, Latitude, Longitude, StartAt, EndAt, Capacity, Status, CreatedAt, UpdatedAt.
- **TicketType** — Τύπος εισιτηρίου με πεδία: ID, EventID, Name, Price, Quantity, Available, CreatedAt, UpdatedAt.
- **Booking** — Κράτηση με πεδία: ID, UserID, EventID, TicketTypeID, Quantity, TotalCost, Status, CreatedAt, UpdatedAt.
- **Message** — Μήνυμα με πεδία: ID, SenderID, RecipientID, EventID (προαιρετικό), Subject, Body, ReadAt, DeletedBySender, DeletedByRecipient, CreatedAt.
- **Category** — Κατηγορία με πεδία: ID, Name (unique), CreatedAt.
- **EventPhoto** — Φωτογραφία εκδήλωσης με πεδία: ID, EventID, StoragePath, Position, CreatedAt.

Επιπλέον, ορίζεται η δομή `EventDetail` για πλούσιο περιεχόμενο εκδήλωσης (συντελεστές, καλλιτέχνες, χρήσιμες πληροφορίες, διοργανωτής, επαναλήψεις), το οποίο αποθηκεύεται ως JSONB και δεν αποτελεί μέρος του DTD εξαγωγής.

4.4.2 Σταθερές (Constants)

Οι σταθερές ορίζονται ως τύποι `string` και χρησιμοποιούν το pattern των const:

- **UserRole**: `admin`, `organizer`, `attendee`, `guest`
- **AccountStatus**: `pending`, `approved`, `rejected`
- **EventStatus**: `draft`, `published`, `cancelled`, `completed` (το `deleted` επιβιώνει μόνο ως παλαιά τιμή σε γραμμές γραμμένες πριν τη λογική διαγραφή και απορρίπτεται από την `ValidEventStatus`)
- **BookingStatus**: `pending`, `confirmed`, `cancelled`

4.4.3 Σφάλματα (Sentinel Errors)

Ορίζονται σφάλματα-φρουροί (sentinel errors) στο `domain/errors.go` που χρησιμοποιούνται σε όλα τα επίπεδα για ομοιόμορφο χειρισμό σφαλμάτων:

Σφάλμα	Περιγραφή
`ErrNotFound`	Το αντικείμενο δεν βρέθηκε
`ErrUnauthorized`	Μη εξουσιοδοτημένη πρόσβαση
`ErrForbidden`	Απαγορευμένη ενέργεια
`ErrUsernameTaken`	Το username είναι ήδη καταχωρημένο
`ErrCapacityExceeded`	Υπέρβαση διαθέσιμων εισιτηρίων
`ErrEventNotBookable`	Η εκδήλωση δεν δέχεται κρατήσεις
`ErrEventNotDeletable`	Η εκδήλωση δεν μπορεί να διαγραφεί
`ErrTicketTypeLocked`	Ο τύπος εισιτηρίου έχει κρατήσεις

Πίνακας 7: Κύρια sentinel errors του domain

4.5 Επίπεδο Αποθετηρίου (Repository Layer)

Το επίπεδο αποθετηρίου βρίσκεται στο `internal/repository/` και ακολουθεί το μοτίβο Dependency Inversion: ορίζονται διασυνδέσεις (interfaces) στο `interfaces.go` και η υλοποίηση γίνεται μέσω GORM στο `internal/repository/postgres/`.

4.5.1 Διασυνδέσεις

Οι κύριες διασυνδέσεις που ορίζονται:

Διασύνδεση	Κύριες μέθοδοι
`UserRepository`	`GetByID`, `GetByUsername`, `GetByEmail`, `Create`, `UpdateProfile`, `Approve`, `Reject`, `SetRole`, `List`, `ListByRole`, `ListByIDs`
`EventRepository`	`Create`, `GetByID`, `ListOrganizer`, `ListPublic`, `Search`, `Update`, `Delete`, `SetStatus`, `MarkCompleted`, `SetCategories`, `SetTicketTypes`, `GetPhotos`, `ListAdmin`, `ListForExport`
`TicketTypeRepository`	`Create`, `GetByID`, `ListByEvent`, `Update`, `Delete`, `DecrementAvailabilityAtomic`

BookingRepository	Create, CreateBookingWithDecrement, GetByID, ListByUser, ListByEvent, ListConfirmedByEvent, ListDistinctConfirmedUserIDsByEvent, ListAllConfirmed, CountByTicketType, ExistsBetweenAttendeeAndOrganizer, ListCorrespondentIDs
MessageRepository	Create, CreateBatch, GetByID, ListInbox, ListSent, MarkAsRead, DeleteForUser, GetUnreadCount
CategoryRepository	Create, GetByID, GetByName, ListAll
SupportRepository	CreateTicket, GetTicket, ListTicketsByUser, ListTickets, CountTicketsByStatus, AddMessage, ListMessages, SetTicketStatus
EventPhotoRepository	Add, ListByEvent, Delete — οι φωτογραφίες της εκδήλωσης
UserEventViewRepository	Record, CountByEvent, ListAll — η καταγραφή προβολών που τροφοδοτεί τον recommender

Πίνακας 8: Διασυνδέσεις του επιπέδου αποθετηρίου

4.5.2 Υλοποίηση με GORM

Τα GORM models ορίζονται στο `postgres/models.go` και οι αντιστοιχίσεις model-προς-domain γίνονται στο `postgres/mappers.go`. Κάθε model χρησιμοποιεί UUID v7 ως πρωτεύον κλειδί.

4.5.3 Ατομικές Λειτουργίες (Atomic Operations)

Η κρίσιμη λειτουργία δημιουργίας κράτησης υλοποιείται με ατομικό τρόπο στο `booking_tx.go`. Η μέθοδος `CreateBookingWithDecrement` εκτελείται εντός μιας συναλλαγής βάσης δεδομένων και ακολουθεί την εξής διαδικασία:

1. Εκτελεί `UPDATE ticket_types SET available = available - $1 WHERE id = $2 AND available >= $1`.
2. Ελέγχει τον αριθμό των γραμμών που επηρεάστηκαν (RowsAffected).
3. Αν `RowsAffected == 0`, ελέγχει αν υπάρχει το ticket type (επιστρέφει `ErrNotFound` αν δεν υπάρχει ή `ErrCapacityExceeded` αν δεν υπάρχουν διαθέσιμες θέσεις).
4. Δημιουργεί την εγγραφή κράτησης (booking) με την υπολογισμένη τιμή `totalCost = price * quantity`.
5. Αν αποτύχει οποιοδήποτε βήμα, γίνεται rollback της συναλλαγής.

Με αυτόν τον τρόπο αποφεύγονται τα race conditions και διασφαλίζεται ότι δύο ταυτόχρονες κρατήσεις δεν θα υπερβούν τη διαθέσιμη χωρητικότητα.

4.5.4 Αναζήτηση Πλήρους Κειμένου

Η αναζήτηση εκδηλώσεων υλοποιείται στο `EventRepository.Search` και υποστηρίζει:

- Αναζήτηση με `ILIKE` στον τίτλο.
- Αναζήτηση ελεύθερου κειμένου με `plainto_tsquery('greek', ...)` στο πεδίο `search_vector` (GIN index).
- Φιλτράρισμα κατά κατηγορία (μέσω του πίνακα `event_categories`).

- Φιλτράρισμα κατά χρονικό διάστημα (overlap check).
- Φιλτράρισμα κατά εύρος τιμής (μέσω EXISTS subquery σε `ticket_types`).
- Keyset pagination: ο πελάτης στέλνει `after_start` και `after_id`, και το `meta` της απόκρισης επιστρέφει `next_after_start` / `next_after_id` για την επόμενη σελίδα.

4.6 Επίπεδο Υπηρεσιών (Service Layer)

Το επίπεδο υπηρεσιών βρίσκεται στο `internal/service/` και περιέχει όλη την επιχειρηματική λογική της εφαρμογής. Κάθε υπηρεσία δέχεται ως εξαρτήσεις τις διασυνδέσεις των αποθετηρίων και άλλες υπηρεσίες.

4.6.1 AuthService

Η υπηρεσία αυθεντικοποίησης διαχειρίζεται την εγγραφή, είσοδο, ανανέωση token και αποσύνδεση χρηστών.

Εγγραφή (`Register`):

1. Επικύρωση ταύτισης password και confirm_password.
2. Έλεγχος μοναδικότητας username και email (επιστροφή `ErrUsernameTaken` ή `ErrEmailTaken` αν υπάρχουν).
3. Κατακερματισμός password με bcrypt και κόστος 12.
4. Δημιουργία χρήστη με τον ρόλο που επέλεξε στη φόρμα (`attendee` ή `organizer`) και κατάσταση `pending`, μέχρι την έγκριση από τον διαχειριστή.

Είσοδος (`Login`):

1. Αναζήτηση χρήστη βάσει username.
2. Σύγκριση bcrypt hash.
3. Έλεγχος κατάστασης λογαριασμού (πρέπει να είναι `approved`).
4. Έκδοση ζεύγους JWT tokens (access + refresh).

JWT Tokens:

- **Access token**: Διάρκεια 15 λεπτά, υπογεγραμμένο με HMAC-SHA256. Claims: `sub` (user ID), `role`, `token_type: "access"`, `jti` (τυχαίο UUID), `exp`, `iat`.
- **Refresh token**: Διάρκεια 7 ημέρες, HMAC-SHA256. Claims: `sub`, `role`, `token_type: "refresh"`, `jti`.
- Το JTI του refresh token αποθηκεύεται στο Redis με κλειδί `auth:refresh:<jti>` και TTL 7 ημερών.

Ανανέωση (`RefreshToken`):

1. Επαλήθευση JWT και έλεγχος `token_type == "refresh"`.
2. Επιβεβαίωση ύπαρξης JTI στο Redis.
3. Διαγραφή παλαιού JTI, έκδοση νέου ζεύγους token.

4.6.2 UserService

Διαχειρίζεται τα προφίλ χρηστών και τις ενέργειες διαχειριστή:

- `ListUsers` — Λίστα χρηστών με σελιδοποίηση.
- `GetUser` — Ανάκτηση στοιχείων χρήστη.
- `ApproveUser` / `RejectUser` — Έγκριση/απόρριψη εγγραφής από διαχειριστή.
- `ChangeRole` — Αλλαγή ρόλου (`attendee` ↔ `organizer`) από διαχειριστή.
- `UpdateProfile` — Μερική ενημέρωση προφίλ (όλα τα πεδία προαιρετικά).

Ο ενσωματωμένος διαχειριστής της Απαίτησης 3 δημιουργείται από τη συνάρτηση `service.EnsureAdmin`, η οποία καλείται σε κάθε εκκίνηση του server: αν δεν υπάρχει κανένας λογαριασμός με ρόλο `admin`, δημιουργείται ένας από τις μεταβλητές `ADMIN_USERNAME` / `ADMIN_PASSWORD` / `ADMIN_EMAIL` (με bcrypt hash και κατάσταση `approved`). Έτσι μια καθαρή εγκατάσταση διαθέτει διαχειριστή χωρίς χειροκίνητο βήμα, ενώ ο seeder (`cmd/seed`) προσθέτει μόνο δοκιμαστικά δεδομένα.

4.6.3 EventService

Η πιο σύνθετη υπηρεσία, διαχειρίζεται ολόκληρο τον κύκλο ζωής των εκδηλώσεων.

Δημιουργία (`Create`):

1. Επικύρωση πεδίων (τίτλος, περιγραφή, ημερομηνίες, χωρητικότητα).
2. Έλεγχος: το άθροισμα των ποσοτήτων όλων των τύπων εισιτηρίων δεν πρέπει να υπερβαίνει τη συνολική χωρητικότητα της εκδήλωσης.
3. Δημιουργία εκδήλωσης με κατάσταση `draft`.
4. Δημιουργία τύπων εισιτηρίων.
5. Επίλυση ή δημιουργία κατηγοριών (μία εκδήλωση μπορεί να ανήκει σε πολλές κατηγορίες).

Δημοσίευση (`Publish`): Μεταβολή κατάστασης από `draft` σε `published`. Επιτρέπει στις κρατήσεις να ξεκινήσουν.

Ακύρωση (`Cancel`): Μεταβολή κατάστασης από `published` σε `cancelled`. Αποστέλλει αυτόματα ενημερωτικό μήνυμα σε όλους τους χρήστες με επιβεβαιωμένες κρατήσεις (μέσω `MessagingService.NotifyEventCancellation`). Νέες κρατήσεις αποκλείονται.

Διαγραφή (`Delete`): Επιτρέπεται μόνο αν η εκδήλωση βρίσκεται σε κατάσταση `draft`, ή αν είναι `published` αλλά δεν έχει καμία κράτηση.

Διαχείριση τύπων εισιτηρίων:

- `AddTicketType`, `UpdateTicketType`, `DeleteTicketType`.
- Έλεγχος χωρητικότητας σε κάθε δημιουργία/ενημέρωση.
- Αποκλεισμός τροποποίησης αν υπάρχουν κρατήσεις για τον συγκεκριμένο τύπο (`ErrTicketTypeLocked`).

Διαχείριση φωτογραφιών:

- Υποστήριξη upload (multipart) και ανίχνευση MIME από bytes (jpeg, png, gif, webp)· το όριο μεγέθους των 5MB ελέγχεται ήδη στον handler.
- Αποθήκευση στη διαδρομή `mediaRoot/<eventID>/<uuid>.<ext>`.
- Διαγραφή και ορισμός featured image.

4.6.4 BookingService

Υπηρεσία διαχείρισης κρατήσεων με τα εξής χαρακτηριστικά:

1. **Ατομικός έλεγχος διαθεσιμότητας** — Χρήση της ατομικής λειτουργίας `CreateBookingWithDecrement` για αποφυγή overbooking.
2. **Υπολογισμός κόστους από τον διακομιστή** — Το `totalCost` υπολογίζεται ως `price * quantity` από την πλευρά του server, χωρίς εμπιστοσύνη στην τιμή που στέλνει ο πελάτης.
3. **Έλεγχος κατάστασης εκδήλωσης** — Η κράτηση επιτρέπεται μόνο για εκδηλώσεις σε κατάσταση `published`.
4. **Πρόσβαση βάσει ρόλου** — Ο διοργανωτής βλέπει τις κρατήσεις των εκδηλώσεών του, ο συμμετέχων βλέπει μόνο τις δικές του.

4.6.5 MessagingService

Η ταχυδρομική θυρίδα της Απαίτησης 10. Ένα μήνυμα είναι μία εγγραφή στον πίνακα `messages`· τα «απεσταλμένα» του αποστολέα και τα «εισερχόμενα» του παραλήπτη είναι η ίδια εγγραφή, φιλτραρισμένη κατά `sender_id` / `recipient_id`.

- `SendMessage` — Επικύρωση θέματος/κειμένου (υποχρεωτικά, με όρια μήκους), έλεγχος ότι οι δύο χρήστες συνδέονται με κράτηση (`canCorrespond`: ο ένας έχει κράτηση σε εκδήλωση του άλλου — τίποτα άλλο δεν ανοίγει το κανάλι), αποθήκευση, ανανέωση του μετρητή μη αναγνωσμένων στο Redis και push των γεγονότων `new_message` και `unread_count` μέσω του Hub.
- `GetInbox` / `GetSent` — Σελιδοποιημένη ανάγνωση φακέλου με συνολικό πλήθος (`meta.total`). Η λίστα **δεν** σημαίνει τίποτα ως αναγνωσμένο, ώστε η ένδειξη «νέο» να παραμένει μέχρι το άνοιγμα.
- `GetMessage` — Άνοιγμα ενός μηνύματος. Μόνο τα δύο μέρη έχουν πρόσβαση και μόνο όσο το μήνυμα βρίσκεται ακόμη στον φάκελό τους. Για τον παραλήπτη θέτει `read_at`, επανυπολογίζει τον μετρητή και τον προωθεί στο WebSocket.
- `DeleteForUser` — Διαγραφή από τον **δικό** του φάκελο (`deleted_by_sender` ή `deleted_by_recipient`)· το αντίγραφο του άλλου μέρους δεν επηρεάζεται.
- `GetUnreadCount` — Cache-aside: Redis (`nexustickets:unread:<userID>`), με επανυπολογισμό από τη βάση σε miss και σε κάθε μεταβολή.
- `ListRecipients` — Οι χρήστες στους οποίους μπορεί να γράψει ο καλών: οι επαφές του από κρατήσεις.

- `NotifyEventAttendees` — Ενημερωτικό μήνυμα του διοργανωτή προς όλους τους χρήστες με επιβεβαιωμένη κράτηση σε εκδήλωσή του (ένα μήνυμα ανά χρήστη, batch INSERT, push σε καθέναν).
- `NotifyEventCancellation` — Η αυτόματη εκδοχή του παραπάνω κατά την ακύρωση εκδήλωσης, με ελληνικό κείμενο και αναφορά στην εκδήλωση (`event_id`).

4.6.6 SupportService

Τα αιτήματα προς τη διαχείριση είναι σκόπιμα ξεχωριστή λειτουργία από τη θυρίδα: ένα **support ticket** έχει έναν ιδιοκτήτη, το χειρίζεται όποιος διαχειριστής απαντήσει και φέρει το δικό του νήμα απαντήσεων, χωρίς να απαιτείται σχέση κράτησης.

- `Open` — Δημιουργία ticket με θέμα και πρώτο μήνυμα, σε μία συναλλαγή, κατάσταση `open`.
- `ListMine` / `ListAll` — Σελιδοποιημένες λίστες για τον χρήστη και για την ουρά της διαχείρισης (με φίλτρο κατάστασης), ταξινομημένες κατά πρόσφατη δραστηριότητα· `Counts` για τον πίνακα ελέγχου.
- `Get` / `Reply` — Ανάγνωση και απάντηση μόνο από τον ιδιοκτήτη ή διαχειριστή. Απάντηση διαχειριστή → `answered`, απάντηση χρήστη → `open`.
- `Close` — Οριστικό κλείσιμο από οποιαδήποτε πλευρά· κλειστό ticket δεν δέχεται απαντήσεις.

4.6.7 RecommendationService

Η υπηρεσία συστάσεων ενσωματώνει τον αλγόριθμο Biased Matrix Factorization (BMF) για την πρόβλεψη ενδιαφέροντος χρήστη-εκδήλωσης. Το μοντέλο εκπαιδεύεται στον **πραγματικό πίνακα αλληλεπιδράσεων της βάσης δεδομένων**: οι επιβεβαιωμένες κρατήσεις (`bookings`, βάρος 3.0) και οι προβολές εκδηλώσεων (`user_event_views`, βάρος 1.0), με κλειδιά τα πραγματικά UUID χρηστών και εκδηλώσεων. Έτσι κάθε πρόταση αντιστοιχεί σε υπαρκτή εκδήλωση της εφαρμογής. Η εκπαίδευση γίνεται μία φορά στην εκκίνηση και έπειτα περιοδικά κάθε 6 ώρες μέσω goroutine ticker, και η αντικατάσταση του μοντέλου γίνεται ατομικά με `sync.RWMutex`. Για χρήστες χωρίς ιστορικό παρέχεται fallback βάσει δημοφιλών εκδηλώσεων (cold-start). Οι μέθοδοι `HasUser`, `ScoreEvents` και `PopularEventIDs` καταναλώνονται από το `EventService`, το οποίο σερβίρει τις τελικές συστάσεις μέσω `GET /events/recommended`. Λεπτομερής περιγραφή του αλγορίθμου δίνεται στο κεφάλαιο «Σύστημα Συστάσεων».

4.6.8 CatalogExportService

Υπηρεσία εξαγωγής δεδομένων σε μορφή XML και JSON, σύμφωνα με το DTD της εκφώνησης.

XML Export:

- Εγγραφή XML header, δήλωσης `<!DOCTYPE Events SYSTEM "events-catalog.dtd">` και ρίζας `<Events>`.
- Ανάγνωση εκδηλώσεων σε batches των 100 εγγραφών.
- Για κάθε εκδήλωση: φόρτωση τύπων εισιτηρίων, κατηγοριών, κρατήσεων, χρηστών.
- Κωδικοποίηση με `xml.Encoder` με indentation.
- Ροή (streaming) του αποτελέσματος απευθείας στον πελάτη χωρίς προσωρινή αποθήκευση όλων των δεδομένων στη μνήμη.

JSON Export:

- Αντίστοιχη λειτουργία με streaming JSON batches και διαχωριστικά κόμματα.
- Μορφή: `{"events":[...]}`.

4.7 Επίπεδο Χειριστών (Handler Layer)

Οι χειριστές (handlers) βρίσκονται στο `internal/handler/` και είναι υπεύθυνοι για την επικοινωνία με τον πελάτη μέσω HTTP. Κάθε handler δέχεται DTOs (Data Transfer Objects) από το `internal/dto/` και ποτέ δεν εκθέτει οντότητες του domain layer απευθείας.

4.7.1 Ομαδοποίηση Διαδρομών (Routes)

Μέθοδος	Διαδρομή	Περιγραφή
POST	`/api/v1/auth/register`	Εγγραφή χρήστη
POST	`/api/v1/auth/login`	Είσοδος
POST	`/api/v1/auth/refresh`	Ανανέωση token
POST	`/api/v1/auth/logout`	Αποσύνδεση
GET	`/api/v1/auth/username-available`	Έλεγχος διαθεσιμότητας username
GET	`/api/v1/users/me`	Στοιχεία χρήστη
PATCH	`/api/v1/users/me`	Ενημέρωση προφίλ
GET	`/api/v1/users/admin`	Λίστα χρηστών
GET	`/api/v1/users/admin/:id`	Στοιχεία χρήστη
PATCH	`/api/v1/users/admin/:id/approve`	Έγκριση χρήστη
PATCH	`/api/v1/users/admin/:id/reject`	Απόρριψη χρήστη
PATCH	`/api/v1/users/admin/:id/role`	Αλλαγή ρόλου
GET	`/api/v1/events/`	Λίστα εκδηλώσεων
GET	`/api/v1/events/search`	Αναζήτηση
GET	`/api/v1/events/recommended`	Προτεινόμενες
GET	`/api/v1/events/:id`	Λεπτομέρειες
GET	`/api/v1/events/organizer/me`	Οι εκδηλώσεις μου
POST	`/api/v1/events/organizer`	Δημιουργία εκδήλωσης
PATCH	`/api/v1/events/organizer/:id`	Ενημέρωση
DELETE	`/api/v1/events/organizer/:id`	Διαγραφή

PATCH	/api/v1/events/organizer/:id/publish	Δημοσίευση
PATCH	/api/v1/events/organizer/:id/cancel	Ακύρωση
POST/PATCH/DELETE	/api/v1/events/organizer/:id/ticket-types[/:ticketTypeId]	Τύποι εισιτηρίων
POST/DELETE	/api/v1/events/organizer/:id/photos[/:photoId]	Φωτογραφίες
POST	/api/v1/events/organizer/:id/featured-image	Εικόνα εξωφύλλου
GET/PATCH	/api/v1/events/admin[/:id]	Διαχείριση εκδηλώσεων από admin
POST	/api/v1/events/admin/:id/featured-image	Εικόνα εξωφύλλου (admin)
POST	/api/v1/bookings/	Δημιουργία κράτησης
GET	/api/v1/bookings/me	Οι κρατήσεις μου
GET	/api/v1/bookings/:id	Μία κράτηση
GET	/api/v1/bookings/organizer/events/:eventId	Κρατήσεις εκδήλωσης (διοργανωτής)
GET	/api/v1/messages/inbox	Εισερχόμενα
GET	/api/v1/messages/sent	Απεσταλμένα
GET	/api/v1/messages/:id	Άνοιγμα μηνύματος (σήμανση ως αναγνωσμένο)
GET	/api/v1/messages/unread-count	Μετρητής μη αναγνωσμένων
GET	/api/v1/messages/recipients	Επιτρεπόμενοι παραλήπτες
POST	/api/v1/messages/	Αποστολή μηνύματος
POST	/api/v1/messages/events/:eventId/notify	Μήνυμα διοργανωτή σε όλους τους συμμετέχοντες
DELETE	/api/v1/messages/:id	Διαγραφή από τον φάκελο του χρήστη
POST/GET	/api/v1/support/tickets[/me]	Νέο αίτημα υποστήριξης / τα αιτήματά μου
GET/POST/PATCH	/api/v1/support/tickets/:id[/messages|/close]	Νήμα, απάντηση, κλείσιμο
GET	/api/v1/support/admin/tickets, /counts	Ουρά αιτημάτων διαχείρισης
GET	/api/v1/categories/	Λίστα κατηγοριών
GET	/api/v1/admin/events/export/xml	Εξαγωγή XML
GET	/api/v1/admin/events/export/json	Εξαγωγή JSON
GET	/api/v1/ws/messages	WebSocket για μηνύματα
GET	/health	Έλεγχος υγείας
GET	/ping	Ping

Πίνακας 9: Διαδρομές REST API

4.7.2 Data Transfer Objects (DTOs)

Όλοι οι χειριστές χρησιμοποιούν DTOs για την επικοινωνία με τον πελάτη:

Request DTOs:

- `RegisterRequest` — username, password, confirm_password, first_name, last_name, email, phone, address, city, country, latitude, longitude, vat_number, role.
- `LoginRequest` — username, password.
- `CreateEventRequest` — title, description, venue, event_type, address, city, country, latitude, longitude, start_at, end_at, capacity, categories, ticket_types, event_detail.
- `CreateBookingRequest` — event_id, ticket_type_id, quantity.
- `SendMessageRequest` — recipient_id, subject, body, event_id (προαιρετικό).
- `NotifyAttendeesRequest` — subject, body.

Response DTOs:

- `AuthTokensResponse` — access_token, refresh_token, token_type, access_expires_in, refresh_expires_in.
- `UserResponse` — Όλα τα πεδία χρήστη εκτός από password hash.
- `EventResponse` — Πλήρη στοιχεία εκδήλωσης με ticket types, categories, photos.
- `BookingResponse` — Στοιχεία κράτησης με event, ticket type, cost.
- `MessageResponse` — Στοιχεία μηνύματος με sender/recipient πληροφορίες, `read_at` (null = μη αναγνωσμένο) και προαιρετική αναφορά εκδήλωσης.

4.7.3 Μορφή API Envelope

Κάθε απόκριση ακολουθεί την ομοιόμορφη μορφή:

```
{
  "data": { ... },
  "error": null,
  "meta": { "total": 53, "limit": 20, "offset": 0 }
}
```

4.8 Ενδιάμεσο Λογισμικό (Middleware)

Το ενδιάμεσο λογισμικό υλοποιείται στο `internal/middleware/` και εφαρμόζεται σε επίπεδο διαδρομής ή ομάδας διαδρομών.

4.8.1 JWTAuth

Το middleware `JWTAuth` επαληθεύει την παρουσία και εγκυρότητα του JWT token:

1. Διαβάζει την κεφαλίδα `Authorization: Bearer <token>`.
2. Αναλύει το JWT, επαληθεύει την υπογραφή HMAC-SHA256.
3. Ελέγχει ότι το `token_type == "access"`.

4. Εξάγει τα claims `sub` (user ID) και `role`.
5. Αποθηκεύει τα στοιχεία στο `c.Locals("user_id")` και `c.Locals("role")`.
6. Σε περίπτωση αποτυχίας, επιστρέφει HTTP 401.

Υπάρχει και η παραλλαγή `OptionalJWTAuth` που αποτυγχάνει σιωπηλά (δεν επιστρέφει σφάλμα αν δεν υπάρχει token), χρησιμοποιείται για διαδρομές όπως `GET /events/recommended` και `GET /events/:id` όπου η αυθεντικοποίηση είναι προαιρετική.

4.8.2 WebSocketJWTAuth

Παραλλαγή του JWTAuth για WebSocket συνδέσεις. Αντί για κεφαλίδα Authorization, διαβάζει το token από το query parameter `?token=<JWT>`.

4.8.3 RequireRole

Το `RequireRole(roles ...domain.UserRole)` είναι ένα guard που ελέγχει αν ο ρόλος του χρήστη (από `c.Locals("role")`) βρίσκεται στο επιτρεπόμενο σύνολο. Αν όχι, επιστρέφει HTTP 403.

4.8.4 RateLimiter

Υλοποίηση sliding window log με χρήση Redis Sorted Sets και Lua script:

1. `ZREMRANGEBYSCORE` — Αφαίρεση καταχωρήσεων εκτός χρονικού παραθύρου.
2. `ZADD` — Προσθήκη τρέχουσας χρονικής στιγμής.
3. `ZCARD` — Μέτρηση καταχωρήσεων εντός παραθύρου.
4. Αν ο αριθμός υπερβαίνει το όριο, επιστροφή HTTP 429.

Το κλειδί στο Redis είναι `rate_limit:<segment>:<IP>`. Διαφορετικά τμήματα (segments) χρησιμοποιούνται για διαφορετικές ομάδες διαδρομών (π.χ. `"auth"` με 5 requests/min, `"global"` με 120 requests/min).

4.8.5 CORS

Επιτρέπει συγκεκριμένα origins που ορίζονται στη μεταβλητή `CORS_ALLOWED_ORIGINS`. Οι επιτρεπόμενες μέθοδοι είναι GET, POST, PUT, PATCH, DELETE, OPTIONS. Υποστηρίζονται credentials και η κεφαλίδα Authorization.

4.8.6 ErrorHandler

Κεντρικός χειριστής σφαλμάτων που αντιστοιχίζει τα sentinel errors του domain σε κατάλληλους HTTP κωδικούς:

Sentinel Error	HTTP Code
`ErrUnauthorized`	401
`ErrForbidden`	403
`ErrNotFound`	404
`ErrUsernameTaken`, `ErrEmailTaken`	409

`ErrMessagingNotAllowed`	403
`ErrCapacityExceeded`	409
`ErrEventNotBookable`, `ErrInvalidEventState`, `ErrTicketTypeLocked`	422
`ErrInvalidRole`, σφάλματα επικύρωσης	400
Άγνωστο σφάλμα	500

Πίνακας 10: Αντιστοίχιση sentinel errors σε HTTP κωδικούς

4.9 WebSocket

Η επικοινωνία πραγματικού χρόνου (Real-time Communication) υλοποιείται μέσω του πρωτοκόλλου WebSocket, χρησιμοποιώντας τη βιβλιοθήκη `gofiber/websocket` (την προσαρμογή του πρωτοκόλλου στο fasthttp που χρησιμοποιεί το Fiber) · από το `gorilla/websocket` δανειζόμαστε μόνο τις σταθερές τύπου πλαισίου (text, ping, close). Η αρχιτεκτονική ακολουθεί το μοτίβο Hub/Client.

4.9.1 Hub

Το `Hub` είναι μια κεντρική δομή που διατηρεί έναν χάρτη συνδέσεων: `map[uuid.UUID]map[*Client]struct{}`. Κάθε χρήστης μπορεί να έχει πολλές ταυτόχρονες συνδέσεις (π.χ. από διαφορετικές καρτέλες του browser).

Η κύρια λειτουργία του Hub είναι ένα event loop που εκτελείται σε ξεχωριστό goroutine και διαχειρίζεται τρία κανάλια:

- `register` — Προσθήκη νέου client στη δομή.
- `unregister` — Αφαίρεση client και κλείσιμο του καναλιού `send`.
- `SendToUser(userID, payload)` — Αποστολή μηνύματος σε όλες τις ενεργές συνδέσεις του χρήστη (non-blocking send).

4.9.2 Client

Κάθε `Client` αντιπροσωπεύει μία WebSocket σύνδεση και διαθέτει:

- `ReadPump` — Αδειάζει τη σύνδεση μέχρι να κλείσει, ώστε να λειτουργεί ο μηχανισμός pong/timeout. Το κανάλι είναι αποκλειστικά server → client: τα μηνύματα αποστέλλονται μέσω REST και τίποτα απ' όσα στέλνει ο πελάτης δεν ερμηνεύεται, γεγονός που απλοποιεί σημαντικά την ασφάλεια.
- `WritePump` — Τικερ ping κάθε 54 δευτερόλεπτα (ping period = 9/10 του pong wait). Γράφει μηνύματα από το κανάλι `send` στη σύνδεση.
- Timeouts: writeWait = 10s, pongWait = 60s, pingPeriod = 54s.
- Μέγιστο μέγεθος εισερχόμενου frame: 1024 bytes.

4.9.3 Ροή Επικοινωνίας

Η ροή επικοινωνίας για τα μηνύματα έχει ως εξής:

1. Ο χρήστης συνδέεται στο `/api/v1/ws/messages?token=<JWT>`.

2. Το `WebSocketJWTAuth` επαληθεύει το JWT token.
3. Το `Hub.Register` αποθηκεύει τον νέο client.
4. Όταν ένας άλλος χρήστης στέλνει μήνυμα μέσω REST (`POST /api/v1/messages/`), η `MessagingService` καλεί `hub.SendToUser(recipientID, payload)`.
5. Το Hub διανέμει το μήνυμα σε όλες τις ενεργές συνδέσεις του παραλήπτη.
6. Ο `WritePump` του client γράφει το μήνυμα στη WebSocket σύνδεση.

Τα τρία είδη μηνυμάτων WebSocket που ορίζονται:

- `new_message` — Νέο μήνυμα στα εισερχόμενα (περιλαμβάνει όλα τα στοιχεία του μηνύματος).
- `unread_count` — Ενημέρωση μετρητή μη αναγνωσμένων μηνυμάτων, σε κάθε μεταβολή του (αποστολή, άνοιγμα, διαγραφή).
- `support_update` — Ειδοποίηση προς την άλλη πλευρά ενός αιτήματος υποστήριξης όταν αυτό ανοίγει, απαντιέται ή κλείνει, ώστε το νήμα και η ουρά της διαχείρισης να ανανεώνονται ζωντανά.

4.10 Εξαγωγή XML και JSON

Η εξαγωγή δεδομένων σε μορφή XML και JSON υλοποιείται στο `internal/export/catalog.go` και ικανοποιεί πλήρως το DTD της εκφώνησης.

4.10.1 XML Export

Η δομή `EventExportXML` αντιστοιχεί ακριβώς στο DTD:

- Κάθε στοιχείο XML αντιστοιχεί σε πεδίο της δομής με XML tags.
- Οι ιδιότητες (attributes) όπως `EventID`, `TicketTypeID`, `BookingID` ορίζονται με `xml:"...,attr"`.
- Οι εμφωλευμένες δομές (TicketTypes, Bookings, Organizer, GeoLocation, Media, Attendee) ορίζονται ως ξεχωριστές δομές.
- Η εξαγωγή γίνεται σε ροή (streaming): ο server διαβάζει τις εκδηλώσεις σε batches (100 εγγραφές ανά batch) και γράφει απευθείας στον HTTP response writer με `SetBodyStreamWriter`.
- Το `Content-Disposition: attachment; filename=events.xml` ενημερώνει τον browser να κατεβάσει το αρχείο.

4.10.2 JSON Export

Αντίστοιχη δομή `EventJSON` με snake_case JSON tags, ίδια λογική streaming με batches.

4.11 Έγχυση Εξαρτήσεων (Dependency Injection)

Η έγχυση εξαρτήσεων γίνεται μέσω του εργαλείου `google/wire`. Στο `internal/wire/wire.go` ορίζονται:

- **RepositorySet** — Όλες οι υλοποιήσεις αποθετηρίων.

- **ServiceSet** — Όλες οι υπηρεσίες, με χρήση provider συναρτήσεων για όσες χρειάζονται ειδική παραμετροποίηση (π.χ. `provideAuthService` που δέχεται `cfg.JWTSecret`).
- **HandlerSet** — Όλοι οι χειριστές.

Η κεντρική συνάρτηση `InitializeApp(db, redis, cfg)` καλεί το `wire.Build` με όλα τα sets και το `wire.Struct(new(AppDependencies), "*")` για να παράγει αυτόματα τη δομή `AppDependencies` με όλες τις εξαρτήσεις.

Το παραγόμενο αρχείο `wire_gen.go` περιέχει την υλοποίηση της συνάρτησης χωρίς αντανακλάσεις (reflection), με στατικό τύπο και βελτιστοποιημένη απόδοση.

5 Υλοποίηση Frontend

Το μετωπιαίο άκρο είναι μια εφαρμογή μίας σελίδας σε **React 19** και **TypeScript** (strict mode), μεταγλωττιζόμενη με Vite. Η διεπαφή είναι δίγλωσση (ελληνικά / αγγλικά, με προεπιλογή τα ελληνικά) και βασίζεται σε Tailwind CSS v4 με συστατικά shadcn/ui (Radix primitives). Η δρομολόγηση γίνεται με React Router v7 και η διαχείριση κατάστασης με Redux Toolkit και RTK Query.

5.1 Δομή Έργου

Κατάλογος	Περιεχόμενο
`src/app/`	Redux store, τα RTK Query API slices (`authApi`, `eventsApi`, `bookingsApi`, `messagesApi`, `usersApi`) και τα slices `auth`, `events`, `messaging`
`src/routes/`	Ο ενιαίος πίνακας διαδρομών με lazy-loaded σελίδες και το `ProtectedRoute`
`src/pages/`	Μία σύνθεση ανά οθόνη: αρχική, εκδηλώσεις, auth, κρατήσεις, `organizer/`, `admin/`, `messages/`
`src/components/`	`ui/` (shadcn), `layout/` (Navbar, AppShell, Footer), και φάκελοι ανά λειτουργία (`events/`, `bookings/`, `messages/`, `admin/`)
`src/hooks/`	`useAuth`, `useWebSocket`
`src/utils/`	`api.ts` (base query με bearer token και αυτόματη ανανέωση), `wsClient.ts` (singleton WebSocket με επανασύνδεση)
`src/types/`	Τα μοντέλα του API, αντίγραφα των DTO του Go
`src/tests/`	Vitest + React Testing Library

Πίνακας 11: Δομή καταλόγων του frontend

5.2 Διεθνοποίηση

Όλα τα κείμενα της διεπαφής βρίσκονται σε δύο αρχεία YAML (`src/locales/gr.yaml`, `src/locales/en.yaml`, περίπου 800 κλειδιά το καθένα, με ταυτόσημη δομή), και κανένα ελληνικό ή αγγλικό κείμενο δεν είναι ενσωματωμένο στα συστατικά. Η συνάρτηση `t(key, params)` του `LangContext` επιλύει ένα κλειδί με τελείες (π.χ. `booking.confirmTitle`) στην τρέχουσα γλώσσα, γεμίζει placeholders της μορφής `{name}` και, αν ένα κλειδί λείπει, επιστρέφει το ίδιο το κλειδί ώστε η παράλειψη να είναι ορατή. Ο κώδικας εκτός React

(μηνύματα επικύρωσης, μορφοποιητές) χρησιμοποιεί την ισοδύναμη `tr()`, που διαβάζει την τρέχουσα γλώσσα από το ίδιο module. Η επιλογή γλώσσας γίνεται από τον επιλογέα της γραμμής πλοήγησης (με σημαίες SVG), αποθηκεύεται στο `localStorage`, ορίζει το `<html lang>` και καθορίζει το locale των `Intl` μορφοποιητών ημερομηνίας και νομίσματος (`el-GR` / `en-GB`).

5.3 Διαχείριση Κατάστασης

Η **κατάσταση διακομιστή** (εκδηλώσεις, κρατήσεις, χρήστες, μηνύματα) ζει αποκλειστικά στο RTK Query. Κάθε endpoint δηλώνει tags (`Event`, `Booking`, `User`, `Message`) και κάθε mutation ακυρώνει τα σχετικά, ώστε οι λίστες να ανανεώνονται αυτόματα μετά από μια αλλαγή χωρίς χειροκίνητο refetch. Η **κατάσταση πελάτη** περιορίζεται σε τρία μικρά slices: τη συνεδρία (χρήστης, access token), τα φίλτρα αναζήτησης και τον μετρητή μη αναγνωσμένων μηνυμάτων.

Ό,τι πρέπει να επιβιώνει σε ανανέωση σελίδας ή να είναι δυνατό να σταλεί ως σύνδεσμος φυλάσσεται στα search params του URL: τα φίλτρα και η σελίδα της αναζήτησης, ο φάκελος / η σελίδα / το ανοιχτό μήνυμα της θυρίδας, καθώς και η πρόθεση σύνταξης μηνύματος (`?compose=1&to=…&event=…`).

5.4 Αυθεντικοποίηση στον Πελάτη

Το `api.ts` περιτυλίγει το `fetchBaseQuery` του RTK Query: προσθέτει την κεφαλίδα `Authorization: Bearer` και, σε απάντηση 401, εκτελεί μία ανανέωση με το refresh token και επαναλαμβάνει το αρχικό αίτημα· αν και η ανανέωση αποτύχει, καθαρίζει τη συνεδρία. Το `ProtectedRoute` ελέγχει σύνδεση και ρόλο και ανακατευθύνει στο `/login` διατηρώντας τον αρχικό προορισμό. Μετά την είσοδο ο διαχειριστής οδηγείται στη σελίδα διαχείρισης χρηστών (`/admin/users`, Απαίτηση 3) και οι υπόλοιποι χρήστες στην αρχική σελίδα (Απαίτηση 5).

5.5 Σελίδες και Απαιτήσεις

Απ.	Σελίδα / συστατικό	Τι υλοποιεί
1	`Home`, `Navbar`	Σελίδα καλωσορίσματος με είσοδο και εγγραφή· πλοήγηση για επισκέπτες
2	`RegisterPage`	Όλα τα πεδία της εκφώνησης με επικύρωση Zod, ζωντανός έλεγχος διαθεσιμότητας username, επιλογή ρόλου, χάρτης για τη γεωγραφική τοποθεσία· οδηγεί στο `PendingApprovalPage`
3–4	`AdminUsersPage`, `AdminUserDetailPage`, `AdminDashboard`	Λίστα χρηστών με φίλτρα και σελιδοποίηση, σελίδα στοιχείων, έγκριση / απόρριψη / αλλαγή ρόλου
5–6	`Home`, `HomeQuickNav`	Αρχική συνδεδεμένου χρήστη με μετάβαση σε διαχείριση ή πλοήγηση εκδηλώσεων

7	`OrganizerEventNewPage`, `OrganizerEventEditPage`, `OrganizerEventsPage`, `EventBookingsPage`, `TicketTypesEditor`	Φόρμα με όλα τα πεδία του DTD, πολλαπλές κατηγορίες, δυναμικοί τύποι εισιτηρίων με ζωντανό έλεγχο χωρητικότητας, φωτογραφίες, δημοσίευση / ακύρωση / διαγραφή με διαλόγους επιβεβαίωσης, επισκόπηση κρατήσεων
8	`EventsIndexPage`, `EventsFilters`, `EventsBrowsePage`	Πλοήγηση ανά κατηγορία και αναζήτηση με ελεύθερο κείμενο, τίτλο, κατηγορίες, χρονικό διάστημα, εύρος τιμής και πόλη· σελιδοποιημένα αποτελέσματα
9	`EventDetailPage`, `EventMapEmbed`, `BookingForm`	Αναλυτικά στοιχεία, χάρτης OpenStreetMap (react-leaflet), επιλογή τύπου/ποσότητας με προεπισκόπηση κόστους και διάλογο επιβεβαίωσης πριν την οριστική κράτηση
10	`MessagesPage`, `MailboxList`, `MessageView`, `ComposeMessageDialog`, `NotifyAttendeesDialog`	Κατάλογοι Εισερχόμενα / Απεσταλμένα, ένδειξη νέων μηνυμάτων, διαγραφή ανά φάκελο, απάντηση, σύνταξη προς επιτρεπόμενους παραλήπτες, μαζικό μήνυμα διοργανωτή
11	δημόσιες διαδρομές	Ο επισκέπτης βλέπει «Σύνδεση για κράτηση» αντί για φόρμα κράτησης
—	`SupportPage`, `AdminSupportPage`, `TicketThread`	Αιτήματα υποστήριξης προς τη διαχείριση (πέραν της εκφώνησης): λίστα, νήμα, απάντηση, κλείσιμο, ουρά διαχειριστή με φίλτρα
12	`AdminEventsPage`	Κουμπιά εξαγωγής XML και JSON (αυθεντικοποιημένη λήψη αρχείου)
13	`Home` («Προτεινόμενα»), `EventsIndexPage`	Ενότητα προτεινόμενων εκδηλώσεων από το `GET /events/recommended`

Πίνακας 12: Αντιστοίχιση απαιτήσεων σε σελίδες του frontend

5.6 Η Θυρίδα Μηνυμάτων

Η σελίδα `/messages` οργανώνεται σε δύο φακέλους, **Εισερχόμενα** και **Απεσταλμένα**, όπως ζητά η Απαίτηση 10. Κάθε φάκελος είναι ένας πίνακας (αποστολέας ή παραλήπτης, θέμα, ημερομηνία) με σελιδοποίηση 20 μηνυμάτων. Τα μη αναγνωσμένα εισερχόμενα εμφανίζονται με έντονη γραφή και μια κουκκίδα, ενώ ο φάκελος φέρει τον συνολικό αριθμό τους. Το άνοιγμα ενός μηνύματος το σημαίνει ως αναγνωσμένο στον server, και ο νέος μετρητής επιστρέφει μέσω WebSocket στο εικονίδιο λογαριασμού, απ' όπου είναι προσβάσιμη η θυρίδα. Ο διάλογος σύνταξης προσφέρει μόνο τους παραλήπτες που επιτρέπει ο server (επαφές από κρατήσεις), ενώ η απάντηση προσυμπληρώνει παραλήπτη, θέμα (`Re:`) και εκδήλωση. Από τη σελίδα «Οι κρατήσεις μου» κάθε κράτηση διαθέτει σύνδεσμο «Μήνυμα στον διοργανωτή», και ο διοργανωτής από τη σελίδα κρατήσεων μιας εκδήλωσης στέλνει ενημέρωση σε όλους τους συμμετέχοντες.

5.7 Ειδοποιήσεις Πραγματικού Χρόνου

Το `useWebSocket`, που τοποθετείται στο `AppShell`, ανοίγει τη σύνδεση μόλις υπάρξει access token και την κλείνει στην αποσύνδεση. Ο μετρητής μη αναγνωσμένων αρχικοποιείται με ένα αίτημα REST και έκτοτε ακολουθεί τα γεγονότα `unread_count`. Κάθε `new_message` ανανεώνει τον φάκελο εισερχομένων αν είναι ανοιχτός και εμφανίζει μια σύντομη ειδοποίηση (toast) με το θέμα, ώστε η άφιξη να γίνεται αντιληπτή σε οποιαδήποτε σελίδα της εφαρμογής. Ο `wsClient` επανασυνδέεται αυτόματα με εκθετική καθυστέρηση αν η σύνδεση πέσει.

6 Αυθεντικοποίηση και Ασφάλεια

Η εκφώνηση απαιτεί κρυπτογράφηση όλων των αλληλεπιδράσεων, αυθεντικοποίηση με JSON Web Tokens και εξουσιοδότηση ανά ρόλο. Το κεφάλαιο συγκεντρώνει πώς ικανοποιούνται τα τρία αυτά, μαζί με τα υπόλοιπα μέτρα ασφάλειας της υλοποίησης.

6.1 Κρυπτογράφηση Μεταφοράς (TLS)

Όλες οι αλληλεπιδράσεις γίνονται μέσω TLS, όπως απαιτεί η Απαίτηση 1. Το backend εκκινεί με `ListenTLS` όταν οριστούν `TLS_CERT_FILE` / `TLS_KEY_FILE`, και αυτά ορίζονται και στα δύο περιβάλλοντα: ο διακομιστής δεν ακούει ποτέ σε καθαρό HTTP. Στην ανάπτυξη ο Vite dev server εξυπηρετεί το SPA με το ίδιο τοπικό πιστοποιητικό mkcert στο `https://localhost:5173` και προωθεί REST και WebSocket στο backend· στην παραγωγή το nginx τερματίζει το TLS στη θύρα 443 και προωθεί με τη σειρά του πάνω από TLS. Έτσι κρυπτογραφείται όχι μόνο το σκέλος browser–nginx αλλά και το εσωτερικό nginx–backend. Η σύνδεση WebSocket ανοίγει πάντα ως `wss://` στην ίδια προέλευση με τη σελίδα, ώστε να μην μπλοκάρεται ως mixed content.

Επειδή τα πιστοποιητικά είναι τοπικά και αυτο-υπογεγραμμένα, το nginx δεν επαληθεύει το πιστοποιητικό του backend (`proxy_ssl_verify off`)· σε πραγματική εγκατάσταση τη θέση τους παίρνουν πιστοποιητικά αναγνωρισμένης αρχής.

6.2 Αυθεντικοποίηση με JSON Web Tokens

Η είσοδος εκδίδει ζεύγος tokens υπογεγραμμένων με HMAC-SHA256:

- **Access token** (15 λεπτά) με claims `sub`, `role`, `token_type`, `jti`, `iat`, `exp`. Συνοδεύει κάθε αίτημα στην κεφαλίδα `Authorization` ή, για το WebSocket, στο query parameter `token`.
- **Refresh token** (7 ημέρες). Το `jti` του αποθηκεύεται στο Redis με TTL· η ανανέωση επαληθεύει την ύπαρξή του, το διαγράφει και εκδίδει νέο ζεύγος (rotation), ενώ η αποσύνδεση το ανακαλεί. Έτσι, παρότι τα JWT είναι stateless, η ανάκληση είναι πραγματική και μια αλλαγή ρόλου ή η απόρριψη λογαριασμού γίνεται αντιληπτή στην επόμενη ανανέωση.

Η ανανέωση γίνεται διαφανώς από τον πελάτη σε απάντηση 401. Οι κωδικοί χρηστών αποθηκεύονται μόνο ως bcrypt hash με κόστος 12 και δεν εμφανίζονται σε κανένα DTO απόκρισης.

6.3 Εξουσιοδότηση ανά Ρόλο

Κάθε ομάδα διαδρομών περνά από `JWTAuth` και `RequireRole(...)`. Οι δημόσιες διαδρομές (λίστα και αναζήτηση εκδηλώσεων, κατηγορίες) δεν απαιτούν καθόλου token, ενώ οι λεπτομέρειες εκδήλωσης και οι προτεινόμενες χρησιμοποιούν `OptionalJWTAuth`, ώστε ο επισκέπτης να εξυπηρετείται χωρίς token ενώ ο συνδεδεμένος χρήστης να αναγνωρίζεται (για την καταγραφή προβολών και για την εξατομίκευση των προτάσεων). Ο έλεγχος ρόλου αφορά την **κατηγορία** ενέργειας· ο έλεγχος **ιδιοκτησίας** (ο διοργανωτής τροποποιεί μόνο τις δικές του εκδηλώσεις, ο χρήστης ανοίγει μόνο δικά του μηνύματα και κρατήσεις) γίνεται στο επίπεδο υπηρεσιών και επιστρέφει `ErrForbidden`.

Ρόλος	**Δικαιώματα**
Επισκέπτης	Πλοήγηση, αναζήτηση, προβολή εκδήλωσης
Συμμετέχων	1. κράτηση, οι κρατήσεις μου, μηνύματα, προφίλ
Διοργανωτής	1. δημιουργία / διαχείριση δικών του εκδηλώσεων, κρατήσεις τους, μαζικό μήνυμα
Διαχειριστής	ό,τι και ο διοργανωτής για δικές του εκδηλώσεις, + διαχείριση χρηστών και ρόλων, επεξεργασία κάθε εκδήλωσης, ουρά αιτημάτων υποστήριξης, εξαγωγή XML / JSON

Πίνακας 13: Δικαιώματα ανά ρόλο

6.4 Λοιπά Μέτρα

- **Επικύρωση εισόδου** σε κάθε όριο: DTO στους handlers, κανόνες πεδίου στις υπηρεσίες, περιορισμοί CHECK στη βάση.
- **Rate limiting** με sliding window στο Redis: 120 αιτήματα/λεπτό ανά IP συνολικά και 5/λεπτό στα endpoints αυθεντικοποίησης, κατά επιθέσεων brute force.
- **Ανέβασμα αρχείων**: ο τύπος εικόνας ανιχνεύεται από τα bytes (όχι από την επέκταση ή την κεφαλίδα του πελάτη), με όριο 5 MB· τα αρχεία αποθηκεύονται με τυχαίο όνομα σε κατάλογο ανά εκδήλωση.
- **Ασφάλεια δεδομένων**: όλα τα ερωτήματα είναι παραμετροποιημένα μέσω GORM· η React κωδικοποιεί εξ ορισμού την έξοδο (XSS)· το CORS περιορίζεται στις δηλωμένες προελεύσεις.
- **WebSocket μονής κατεύθυνσης**: ο server δεν ερμηνεύει τίποτα απ' όσα στέλνει ο πελάτης, οπότε δεν υπάρχει επιφάνεια επίθεσης μέσω του καναλιού.
- **Μυστικά**: κανένα μυστικό στον κώδικα· όλα προέρχονται από μεταβλητές περιβάλλοντος, με `.env.example` ως πρότυπο και το `.env` εκτός git.

7 Σύστημα Συστάσεων

Η Απαίτηση 13 της εκφώνησης ζητά την προβολή προτεινόμενων εκδηλώσεων βάσει του ιστορικού κρατήσεων και των εκδηλώσεων που έχει επισκεφθεί ο χρήστης, με χρήση του αλγορίθμου **Biased Matrix Factorization (BMF)** υλοποιημένου εκ του μηδενός.

7.1 Ο Αλγόριθμος BMF

Το BMF είναι μια τεχνική παραγοντοποίησης πίνακα (matrix factorization) που προβλέπει την προτίμηση ενός χρήστη u για μια εκδήλωση i ως:

$$\hat{R}(u,i) = \mu + b_u + b_i + P_u \cdot Q_i^T$$

όπου:

- μ — ο καθολικός μέσος όρος όλων των παρατηρούμενων βαθμολογιών,
- b_u — η προκατάληψη (bias) του χρήστη u,
- b_i — η προκατάληψη της εκδήλωσης i,
- P_u, Q_i — τα διανύσματα λανθανόντων παραγόντων (latent factors) διάστασης $K = 20$.

Οι παράμετροι μαθαίνονται με **Stochastic Gradient Descent (SGD)**. Για κάθε παρατήρηση υπολογίζεται το σφάλμα $e_{ui} = R(u,i) - \hat{R}(u,i)$ και οι παράμετροι ενημερώνονται με ρυθμό μάθησης $\eta = 0.01$ και συντελεστή κανονικοποίησης (regularization) $\lambda = 0.02$:

$$b_u \leftarrow b_u + \eta(e_{ui} - \lambda b_u)$$

$$b_i \leftarrow b_i + \eta(e_{ui} - \lambda b_i)$$

$$P_u \leftarrow P_u + \eta(e_{ui} Q_i - \lambda P_u)$$

$$Q_i \leftarrow Q_i + \eta(e_{ui} P_u - \lambda Q_i)$$

Η εκπαίδευση εκτελείται για 30 epochs, με τυχαία ανακάτεψη (shuffle) των παρατηρήσεων σε κάθε epoch. Η υλοποίηση βρίσκεται στο `internal/service/recommendation_engine.go` (`trainBMF`, `BMFFactors.Predict`) και δεν χρησιμοποιεί καμία εξωτερική βιβλιοθήκη μηχανικής μάθησης.

7.2 Δεδομένα Εκπαίδευσης — Έμμεση Ανατροφοδότηση

Καθώς η εφαρμογή δεν διαθέτει ρητές βαθμολογίες (explicit ratings), το μοντέλο εκπαιδεύεται σε **έμμεση ανατροφοδότηση (implicit feedback)** που εξάγεται απευθείας από τη βάση δεδομένων:

Σήμα	Πηγή	Βάρος
Επιβεβαιωμένη κράτηση (ισχυρό)	`bookings`	3.0
Προβολή εκδήλωσης (ασθενές)	`user_event_views`	1.0

Πίνακας 14: Σήματα έμμεσης ανατροφοδότησης και τα βάρη τους

Για κάθε ζεύγος (χρήστης, εκδήλωση) διατηρείται το ισχυρότερο σήμα. Επειδή τα κλειδιά είναι τα πραγματικά UUID της βάσης, κάθε πρόβλεψη αντιστοιχεί σε υπαρκτή εκδήλωση. Η `RecommendationService.train` χτίζει τον πίνακα αλληλεπιδράσεων, εκπαιδεύει το μοντέλο και το εναλλάσσει ατομικά (`sync.RWMutex`). Η διαδικασία επαναλαμβάνεται στην εκκίνηση και κάθε 6 ώρες.

7.3 Παρουσίαση Συστάσεων

Το endpoint `GET /events/recommended` (με προαιρετικό JWT) επιστρέφει πλήρη αντικείμενα εκδήλωσης ως εξής:

1. Ανάκτηση ενός συνόλου δημοσιευμένων εκδηλώσεων.
2. Αποκλεισμός των εκδηλώσεων που ο χρήστης έχει ήδη κρατήσει.
3. Ταξινόμηση των υπολοίπων κατά φθίνουσα βαθμολογία BMF.
4. **Cold-start:** για επισκέπτη, για μη εκπαιδευμένο μοντέλο ή για χρήστη χωρίς ιστορικό, επιστρέφεται η σειρά δημοφιλίας (περισσότερες επιβεβαιωμένες κρατήσεις πρώτα). Αυτό ικανοποιεί την απαίτηση οι χρήστες χωρίς ιστορικό κρατήσεων να εξυπηρετούνται βάσει όσων έχουν δει και όσων είναι δημοφιλή.

7.4 Αξιολόγηση με το Παρεχόμενο Σύνολο Δεδομένων

Το σύνολο δεδομένων που παρέχεται με την εκφώνηση (Kaggle *Event Recommendation Engine Challenge*) χρησιμοποιείται για την **offline αξιολόγηση** της υλοποίησης του BMF, εκτός της ροής παραγωγής (τα ακέραια αναγνωριστικά του δεν αντιστοιχούν σε εκδηλώσεις της εφαρμογής). Ο έλεγχος `TestBMFOfflineEvaluation` χτίζει έναν πίνακα έμμεσης ανατροφοδότησης από το `event_interest.csv` (`interested` $\rightarrow 1.0$, `not_interested` $\rightarrow 0.0$), εφαρμόζει διαχωρισμό train/test 80/20, εκπαιδεύει το μοντέλο και αναφέρει το **RMSE** στο σύνολο ελέγχου, το οποίο κυμαίνεται περίπου στο **0.19** — σημαντικά καλύτερο από την πρόβλεψη σταθεράς. Ο έλεγχος παρακάμπτεται αυτόματα όταν το σύνολο δεδομένων δεν είναι διαθέσιμο, ώστε να μη μπλοκάρει τη συνήθη εκτέλεση των δοκιμών.

8 Δοκιμές

Η στρατηγική δοκιμών εστιάζει στους επιχειρηματικούς κανόνες, εκεί όπου ένα λάθος θα παραβίαζε απαίτηση της εκφώνησης, και συμπληρώνεται από δοκιμές συστατικών στο frontend και χειροκίνητη επαλήθευση από άκρο σε άκρο.

8.1 Backend

Οι δοκιμές του Go γράφονται με το τυπικό πακέτο `testing` και εκτελούνται με `go test ./...` (153 περιπτώσεις). Οι υπηρεσίες ελέγχονται σε απομόνωση με in-memory υλοποιήσεις των διασυνδέσεων αποθετηρίου, ώστε κάθε κανόνας να ελέγχεται χωρίς βάση δεδομένων· μόνο το Redis είναι πραγματικό, επειδή η λογική των tokens και των μετρητών εξαρτάται από τη σημασιολογία του.

Ενδεικτικά καλύπτονται:

- **Αυθεντικοποίηση**: επικύρωση εγγραφής (κωδικοί, email, ΑΦΜ, ρόλος), απόρριψη διπλού username, είσοδος μόνο για εγκεκριμένους λογαριασμούς, rotation και ανάκληση refresh token, ενημέρωση ρόλου στην ανανέωση.
- **Εκδηλώσεις**: άθροισμα ποσοτήτων εισιτηρίων $\leq$ χωρητικότητα σε δημιουργία και ενημέρωση, απαγόρευση μείωσης ποσότητας κάτω από τις πωλημένες θέσεις, κανόνες διαγραφής και ακύρωσης, προϋποθέσεις δημοσίευσης, απόκρυψη προσχεδίων από τρίτους, μετάβαση σε `completed`.
- **Κρατήσεις**: επιτυχής δέσμευση και ενημέρωση διαθεσιμότητας, απόρριψη όταν δεν επαρκούν οι θέσεις (`ErrCapacityExceeded`), απόρριψη για μη δημοσιευμένη ή ληγμένη εκδήλωση, υπολογισμός κόστους από τον server.
- **Μηνύματα**: κανόνας κράτησης μεταξύ των δύο μερών (και το ότι ο διαχειριστής **δεν** εξαιρείται από αυτόν, αφού τα αιτήματα προς τη διαχείριση περνούν από τα support tickets), αρχειοθέτηση στους δύο φακέλους, σελιδοποίηση, το ότι η λίστα δεν σημαίνει ανάγνωση ενώ το άνοιγμα ναι, ανεξάρτητη διαγραφή ανά φάκελο, μετρητής μη αναγνωσμένων με fallback στη βάση, ειδοποίηση ακύρωσης και μαζικό μήνυμα διοργανωτή (ένα ανά χρήστη, όχι στον ίδιο τον διοργανωτή).
- **Συστάσεις**: εκπαίδευση και πρόβλεψη του BMF, cold-start, και η offline αξιολόγηση στο παρεχόμενο dataset (RMSE ≈ 0.19).
- **Εξαγωγή**: η δομή XML αντιστοιχεί στο DTD (σειρά στοιχείων, attributes, προαιρετικά στοιχεία), μετατροπές τιμών, JSON με την ίδια δομή.
- **Υποδομή**: φόρτωση και επικύρωση διαμόρφωσης, middleware (JWT, ρόλοι, rate limiter, αντιστοίχιση σφαλμάτων), WebSocket Hub (εγγραφή, broadcast σε πολλαπλές συνδέσεις, αποχώρηση), mappers μοντέλων, δημιουργία του ενσωματωμένου διαχειριστή.

Επιπλέον, η εξαγωγή XML του πλήρους καταλόγου εκδηλώσεων της πραγματικής βάσης (με κρατήσεις και φωτογραφίες) επικυρώθηκε ως προς το `docs/events-catalog.dtd` με έναν validating parser (lxml) χωρίς σφάλματα.

8.2 Frontend

Οι δοκιμές του frontend γράφονται με Vitest και React Testing Library σε περιβάλλον happy-dom (137 περιπτώσεις σε 17 αρχεία). Οι σελίδες αποδίδονται με τους πραγματικούς providers (store, router) και ένα stub του `fetch` που απαντά ανά διαδρομή, ώστε να ελέγχεται τι ακριβώς ζητά η σελίδα από το API και πώς αποδίδει την απάντηση. Καλύπτονται η είσοδος (συμπεριλαμβανομένης της ανακατεύθυνσης του διαχειριστή), η αναζήτηση και τα φίλτρα με συγχρονισμό URL, η σελίδα εκδήλωσης και η φόρμα κράτησης με τον διάλογο επιβεβαίωσης, οι κρατήσεις μου, η διαχείριση χρηστών, η θυρίδα μηνυμάτων (φάκελοι, σελιδοποίηση, ένδειξη μη αναγνωσμένων, άνοιγμα, απάντηση, διαγραφή, σύνταξη με τους

επιτρεπόμενους παραλήπτες και τα μηνύματα σφάλματος), οι ενότητες της αρχικής σελίδας και βοηθητικές μονάδες (envelope, slices, λήψη εξαγωγής).

8.3 Στατικοί Έλεγχοι και Ολοκλήρωση

Ο κώδικας περνά από `go vet`, `tsc --noEmit` (strict) και ESLint (με τους κανόνες του React Compiler). Η ροή `ci.yml` του GitHub Actions εκτελείται σε κάθε pull request και σε κάθε push στον `main` με τρεις εργασίες: `backend` (`gofmt`, `go vet`, έλεγχος ότι το παραγόμενο `wire_gen.go` είναι ενημερωμένο και `go test ./...` με πραγματικό Redis service), `frontend` (`tsc --noEmit`, ESLint, Vitest και παραγωγικό build) και `docker` (χτίσιμο και των δύο εικόνων). Μια δεύτερη ροή, η `submission.yml`, εκτελείται χειροκίνητα και παράγει τα δύο παραδοτέα: το `report.pdf` μέσω Typst και το zip του πηγαίου κώδικα μέσω `git archive`. Τέλος, η πλήρης ροή — εγγραφή, έγκριση, δημιουργία και δημοσίευση εκδήλωσης, κράτηση, μήνυμα και λήψη του μέσω WebSocket, ακύρωση με ειδοποίηση, εξαγωγή — επαληθεύθηκε χειροκίνητα στο πλήρες Docker περιβάλλον μέσω `https://localhost:5173`.

9 Επίλογος

Η εργασία υλοποίησε την εφαρμογή NexusTickets, μια πλήρη πλατφόρμα διαχείρισης εκδηλώσεων και ηλεκτρονικών κρατήσεων στον Παγκόσμιο Ιστό, σύμφωνα με τις απαιτήσεις της εκφώνησης. Η ανάπτυξη οργανώθηκε σε τρία μέρη: τη σχεδίαση της σχεσιακής βάσης δεδομένων με αφετηρία το DTD της εκφώνησης, την υλοποίηση του REST API σε Go με σαφή διαχωρισμό επιπέδων (domain, repositories, services, handlers, middleware) και την υλοποίηση της εφαρμογής μίας σελίδας σε React/TypeScript. Παράλληλα αναπτύχθηκαν το κανάλι ειδοποιήσεων WebSocket, ο μηχανισμός εξαγωγής XML/JSON και το σύστημα συστάσεων με τον αλγόριθμο Biased Matrix Factorization. Κάθε λειτουργική απαίτηση συνοδεύεται από αυτοματοποιημένες δοκιμές στο επίπεδο όπου ζει ο αντίστοιχος κανόνας.

9.1 Δυσκολίες και Λύσεις

Υπερκράτηση υπό ταυτόχρονη πρόσβαση. Η προφανής προσέγγιση «ανάγνωση διαθεσιμότητας, έλεγχος, εγγραφή» είναι ευάλωτη σε race condition όταν δύο χρήστες κάνουν κράτηση την ίδια στιγμή. Το πρόβλημα αντιμετωπίστηκε με τη συγχώνευση ελέγχου και μείωσης σε μία ατομική πρόταση SQL (`UPDATE … WHERE available >= n`) εντός της ίδιας συναλλαγής με την εισαγωγή της κράτησης· το πλήθος των επηρεαζόμενων γραμμών αποτελεί το κριτήριο επιτυχίας, χωρίς κλειδώματα σε επίπεδο εφαρμογής.

Ανάκληση JSON Web Tokens. Τα JWT από τη φύση τους δεν ανακαλούνται, ενώ η εφαρμογή χρειάζεται πραγματική αποσύνδεση, απόρριψη λογαριασμού και αλλαγή ρόλου. Η λύση συνδυάζει access tokens σύντομης διάρκειας με refresh tokens των οποίων το `jti` καταχωρίζεται στο Redis με TTL και εναλλάσσεται (rotation) σε κάθε ανανέωση, ώστε η κατάσταση του λογαριασμού να ελέγχεται σε κάθε ανανέωση.

Αναζήτηση πλήρους κειμένου στα ελληνικά. Η αναζήτηση με `ILIKE` δεν αντιμετωπίζει την κλίση των ελληνικών λέξεων. Χρησιμοποιήθηκε η διαμόρφωση `greek` της PostgreSQL σε μια γεννώμενη στήλη `tsvector` με δείκτη GIN, που προσφέρει σωστά αποτελέσματα με σταθερή απόδοση χωρίς εξωτερική μηχανή αναζήτησης.

TLS και WebSocket από μία προέλευση σε περιβάλλον ανάπτυξης. Η εφαρμογή τρέχει εξ ολοκλήρου μέσω TLS ήδη στην ανάπτυξη, με το SPA και το API στην ίδια προέλευση. Η χειραψία WebSocket απαιτεί HTTP/1.1, ενώ ο Vite dev server ανοίγει τον HTTPS διακομιστή του μέσω του `node:http2`· η υλοποίηση του Bun για το `node:http2` δεν υποστηρίζει την υποβάθμιση σε HTTP/1.1, με αποτέλεσμα οι σελίδες να φορτώνουν αλλά η σύνδεση `wss://` να μην ολοκληρώνεται. Για τον λόγο αυτό ο Vite εκτελείται με Node, το runtime για το οποίο έχει σχεδιαστεί, ενώ το Bun χρησιμοποιείται ως διαχειριστής πακέτων και εκτελεστής δοκιμών.

Σημασιολογία της θυρίδας μηνυμάτων. Η εκφώνηση ζητά καταλόγους εισερχομένων και απεσταλμένων με ανεξάρτητη διαγραφή ανά κατάλογο, ένδειξη νέων μηνυμάτων και μαζική ενημέρωση από τον διοργανωτή. Το σχήμα μίας γραμμής ανά μήνυμα με δύο σημαίες διαγραφής ικανοποιεί και τα τρία χωρίς αντιγραφή δεδομένων, ενώ ο διαχωρισμός «η λίστα δεν σημαίνει ανάγνωση, το άνοιγμα ναι» κρατά την ένδειξη «νέο» ακριβή. Το WebSocket χρησιμοποιείται αποκλειστικά ως κανάλι ώθησης από τον server προς τον πελάτη, ώστε η αποθήκευση να περνά πάντα από το REST API και τους ελέγχους του.

Συστάσεις χωρίς ρητές βαθμολογίες. Ο αλγόριθμος BMF προϋποθέτει βαθμολογίες, ενώ η εφαρμογή διαθέτει μόνο κρατήσεις και προβολές. Ορίστηκε ένας πίνακας έμμεσης ανατροφοδότησης με σταθμισμένα σήματα (κράτηση 3.0, προβολή 1.0), ώστε ο χρήστης χωρίς κρατήσεις να εξυπηρετείται από τις προβολές του, όπως απαιτεί η εκφώνηση. Το παρεχόμενο σύνολο δεδομένων αξιοποιείται για την offline επαλήθευση της ορθότητας της υλοποίησης, εκτός της ροής παραγωγής.

9.2 Αποτίμηση

Η εφαρμογή καλύπτει τις δεκατρείς λειτουργικές απαιτήσεις της εκφώνησης καθώς και τις τεχνικές: REST API, αντιστοίχιση του σχεσιακού μοντέλου σε αντικειμενοστρεφές, JavaScript/TypeScript framework στο μετωπιαίο άκρο, αυθεντικοποίηση με JWT και εξουσιοδότηση ανά ρόλο, κρυπτογράφηση όλων των αλληλεπιδράσεων με TLS. Το σύστημα εκκινεί από μία εντολή σε καθαρό περιβάλλον, με τον ενσωματωμένο διαχειριστή διαθέσιμο από την πρώτη εκκίνηση, και συνοδεύεται από 147 δοκιμές στο backend και 137 στο frontend.